Контест 6 · Игровой цикл и рандом

Неделя 12 · темы недель 11–12: случайность и перемешивание, конечные автоматы, столкновения, фиксированный шаг, интерполяция, повторы. Сдача устная у экрана: показываете код, запускаете на примерах, объясняете идею.

Монеты: обычная задача — **2**, со звёздочкой (★, например B4*) — **4**, каждая часть босса — **6**. В этом контесте: 15 обычных, 3 со звёздочкой, 3 частей босса — всего **60** монет. Задача засчитывается, только если вы можете объяснить свой код.

A. Рандом

A1. `fisher_yates_trace(n, rolls)` — перемешайте `[0, 1, …, n−1]` алгоритмом Фишера — Йетса: для `i` от `n−1` до `1` меняем местами `a[i]` и `a[j]`, где `j` — очередной бросок из `rolls` (`0 ≤ j ≤ i`). Бросков не `n − 1` или бросок вне диапазона — `ValueError`.

```
Ввод:  fisher_yates_trace(4, [1, 2, 0])
Вывод: [3, 0, 2, 1]
```

A2. `weighted_pick(weights, r)` — лутбокс. Веса — словарь в порядке «редкости», `r` — целое из `[0, сумма)`: первые w_1 значений дают первый предмет, следующие w_2 — второй и т. д. `r` вне диапазона — `ValueError`.

```
Ввод:  W = {"common": 70, "rare": 25, "epic": 5}
       weighted_pick(W, 69), weighted_pick(W, 70), weighted_pick(W, 95)
Вывод: "common", "rare", "epic"
```

A3. `lcg(seed, n, a=1103515245, c=12345, m=2**31)` — `n` следующих чисел линейного конгруэнтного генератора `x ← (a·x + c) mod m`. Одинаковое зерно — одинаковая игра на сервере и в повторе.

```
Ввод:  lcg(1, 3), [x % 6 + 1 for x in lcg(42, 5)]
Вывод: [1103527590, 377401575, 662824084], [4, 3, 6, 5, 2]
```

A4*. `naive_shuffle_counts(n)` — «наивное» перемешивание: для каждого `i` от `0` до `n−1` меняем `a[i]` с `a[j]`, где `j` — **любой** из `0..n−1`. Переберите все n^n вариантов бросков и посчитайте, сколько раз выходит каждая перестановка. Ответ — словарь `перестановка → число`, ключи по возрастанию. Объясните, почему так нельзя раздавать карты.

```
Ввод:  naive_shuffle_counts(3)
Вывод: {(0, 1, 2): 4, (0, 2, 1): 5, (1, 0, 2): 5, (1, 2, 0): 5, (2, 0, 1): 4, (2, 1, 0): 4}
```

B. Конечные автоматы

Автомат — словарь `(состояние, событие) → новое состояние`. Пример для всех задач B:

```
M = {("lobby", "ready"): "countdown", ("countdown", "cancel"): "lobby", ("countdown", "go"):
"playing",
     ("playing", "win"): "finished", ("playing", "pause"): "paused", ("paused", "resume"):
"playing",
     ("paused", "quit"): "limbo", ("limbo", "tick"): "limbo", ("finished", "again"):
"lobby",
     ("ghost", "x"): "lobby"}
```

B1. `run_fsm(trans, start, events)` — пропустите события через автомат. Событие без перехода игнорируется (как сервер игнорирует «ход» в лобби). Ответ — `(итоговое состояние, сколько событий проигнорировано)`.

```
Ввод:  run_fsm(M, "lobby", ["ready", "go", "win", "go"])
Вывод: ("finished", 1)
```

B2. `reachable(trans, start)` — все состояния, достижимые из `start` (и само `start`), по имени.

```
Ввод:  reachable(M, "lobby")
Вывод: ["countdown", "finished", "limbo", "lobby", "paused", "playing"]
```

B3. `dead_states(trans, start, finals)` — достижимые состояния, из которых уже **нельзя** попасть ни в одно из `finals`: игрок там застрянет навсегда. По имени.

```
Ввод:  dead_states(M, "lobby", {"finished"})
Вывод: ["limbo"]
```

C. Столкновения

Прямоугольник — `(x, y, w, h)`, ось `y` вниз. Пример для C1 и C4*:

```
B = {"hero": (0, 0, 4, 4), "wall": (4, 0, 2, 10), "coin": (3, 3, 1, 1),
     "enemy": (10, 10, 2, 2), "bush": (2, -3, 1, 4)}
```

C1. `aabb_pairs(boxes)` — все пары пересекающихся прямоугольников. Касание сторон **не** считается (у пересечения должна быть площадь). Пары `(меньшее имя, большее)`, список по возрастанию.

```
Ввод:  aabb_pairs(B)
Вывод: [("bush", "hero"), ("coin", "hero")]        # hero и wall только касаются
```

C2. `circle_pairs(circles)` — то же для кругов `(x, y, r)`, но касание **считается**. Без корней и `float`.

```
Ввод:  circle_pairs({"a": (0, 0, 2), "b": (3, 4, 3), "c": (10, 0, 1)})
Вывод: [("a", "b")]
```

C3. `segment_hits_rect(p, q, rect)` — задевает ли отрезок `p → q` (полёт пули за тик) замкнутый прямоугольник. Ответ — `True` / `False` без ошибок округления (подсказка: `fractions.Fraction`).

```
Ввод:  segment_hits_rect((0, 0), (10, 4), (4, 0, 2, 2)), segment_hits_rect((0, 0), (10, 10),
(4, 0, 2, 2))
Вывод: True, False
```

C4*. `spatial_hash_pairs(boxes, cell)` — тот же ответ, что в C1, но быстро: прямоугольник попадает во все клетки сетки от `(⌊x/cell⌋, ⌊y/cell⌋)` до `(⌊(x+w)/cell⌋, ⌊(y+h)/cell⌋)` включительно; проверяем только пары, у которых есть общая клетка. Ответ — `(пары, сколько разных пар проверено)`.

```
Ввод:  spatial_hash_pairs(B, 4)
Вывод: ([("bush", "hero"), ("coin", "hero")], 5)
```

D. Время и углы

D1. `lerp_angle(a, b, t)` — поворот башни от угла `a` к `b` (градусы) по **короткой** дуге; ответ в `[0, 360)`. Ровно 180° — поворот в положительную сторону.

```
Ввод:  lerp_angle(350, 10, 0.5), lerp_angle(10, 350, 0.25), lerp_angle(0, 180, 0.5)
Вывод: 0.0, 5.0, 90.0
```

D2. `fixed_steps(frame_times, step, max_steps)` — моменты кадров (мс). Физика идёт шагами по `step` мс: время копится, на каждом кадре делаем столько шагов, сколько помещается. Больше `max_steps` за кадр нельзя («спираль смерти»): тогда делаем `max_steps`, а в копилке оставляем только остаток от деления на `step`. Ответ — шаги на каждом кадре, начиная со второго.

```
Ввод:  fixed_steps([0, 16, 33, 50, 100, 300], 20, 3)
Вывод: [0, 1, 1, 3, 3]
```

D3. `frame_stats(frame_times)` — `(fps, худший кадр, рывков)`: `fps` — кадров в секунду (интервалов на всё время, округлить), худший кадр — наибольший интервал, рывок — интервал больше двух медиан (медиана — нижняя средняя при чётном числе). Меньше двух кадров — `ValueError`.

```
Ввод:  frame_stats([0, 16, 33, 50, 66, 116, 133, 150, 166, 183])
Вывод: (49, 50, 1)
```

E. Проект и алгоритмы

E1. Игра 2 запускается на двух телефонах в одной комнате: лобби, отсчёт, партия, итог. Покажите, что переподключение одного игрока не ломает партию второму.

E2. Клиент на `requestAnimationFrame`: интерполяция состояний (лаба 12), цвета из темы Telegram (`--tg-theme-*`), кнопка «Назад» и `MainButton` работают. Покажите движение с включённым в DevTools троттлингом сети.

E3. Мини-исследование: можно ли писать клиент игры на Python (Pyodide, Brython) вместо JS? Замерьте размер загрузки и время старта в Telegram на телефоне, сравните с вашим JS-клиентом, сделайте вывод на одном слайде.

E4 · босс. Повторы партий: сервер пишет входы игроков, а клиент по ним воспроизводит партию. За каждую часть свой плюс.

а) `rle_encode(inputs)` / `rle_decode(runs)` — входы по тикам (`None` — ничего не нажато) сжимаем в серии `[значение, длина]`. Серия длины `≤ 0` при раскодировании — `ValueError`.

```
Ввод:  rle_encode([None, None, "up", "up", "up", None, "left"])
Вывод: [[None, 2], ["up", 3], [None, 1], ["left", 1]]
```

б) `delta(prev, cur)` и `apply_delta(state, d)` — вместо полного состояния шлём разницу: что поменялось или появилось (`"set"`) и какие ключи исчезли (`"del"`, по возрастанию). `apply_delta` не меняет `state`.

```
Ввод:  delta({"x": 1, "y": 2, "hp": 5}, {"x": 2, "y": 2, "shield": True})
Вывод: {"set": {"x": 2, "shield": True}, "del": ["hp"]}
```

в) `first_desync(n, check)` — повтор разошёлся с сервером. `check(i)` сравнивает хэши состояния на тике `i` (`0 ≤ i < n`): сначала все `True`, потом все `False`. Найдите первый `False` (или `None`), вызвав

`check` не больше $\lfloor \log_2 n \rfloor + 2$ раз.

```
Ввод:  first_desync(1000, lambda i: i < 637)
Вывод: 637
```

E5*. `time_of_impact(a, v, b)` — прямоугольник `a` за тик сдвигается на `v = (vx, vy)`, `b` стоит на месте. Найдите наименьшее `t ∈ [0, 1]`, когда они коснутся (касание считается; уже пересекаются — `0`), или `None`. Так быстрая пуля не пролетает сквозь тонкую стену.

```
Ввод:  time_of_impact((0, 0, 2, 2), (10, 0), (5, 0, 2, 2)), time_of_impact((0, 0, 2, 2), (2,
0), (5, 0, 2, 2))
Вывод: 0.3, None
```